

Intermediate (build the loop yourself) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, and what a real agent loop needs beyond the happy path). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You already know how to make one tool call and get one result back (Doc06). An agent loop just repeats that: call the model, if it asks for a tool run it and feed the result back in, ask the model again — over and over — until it stops asking for tools and just answers.

Things to use: - A `while` or `for` loop with a step counter, and a max-steps number so it can't run forever. - One tool registered, with a Pydantic model for its arguments (from Doc06). - Each round: call the model with the tools list and the messages so far. - If the reply has a tool call: run the tool, add the result to messages. - If not: that's your final answer, stop the loop.

Intermediate Version

The loop's state is just the growing `messages` list (the "scratchpad" from Core Concepts) — each round appends the model's tool-call request, then your tool's result, before calling the model again. The loop ends when a model response has no `tool_calls` in it — that's your signal it decided to answer directly instead of asking for another tool.

The exact pieces: - `for step in range(max_iterations):` — bounding the loop from the very first line, not as an afterthought (Core Concepts' "the limit isn't optional" point, now actually in code). - `response = client.chat.completions.create(model=..., messages=messages, tools=[tool_schema])` each round. - `if response.choices[0].message.tool_calls:` — checking specifically for tool calls, not just "did it reply." - Append the assistant's tool-call message to `messages`, then append a `{"role": "tool", "tool_call_id": ..., "content": result}` message with your tool's result — the API requires both, in that order, or the next call will error. - `else:` — no tool calls means this is the final answer; return `response.choices[0].message.content` and break out of the loop.

Two things the happy path above skips over, worth building in from the start: **a response can request more than one tool call in the same round** — `message.tool_calls` is a list, and you must loop over all of it, appending a `{"role": "tool", ...}` message for every call before asking the model again, or the next API call will error with a mismatched tool response. **A tool can raise an exception** — `run_tool()` calling your real Python function directly, unguarded, means one bad argument or a network hiccup crashes the entire agent, losing every step already completed. Think about what `run_tool()` should do instead: catch the exception, and feed the *error message itself* back in as the Observation, exactly like a tool that returned a normal (if

unhelpful) result — this is what lets the model see the failure and react to it, instead of your program just dying.

Difference between Basic and Intermediate: Basic names the tools and the loop's shape for a single tool that always succeeds. Intermediate shows the real Python syntax, the exact message-passing contract the API expects, and the 2 things that only matter once a response can request multiple tools at once, or a tool can actually fail mid-run — which is the difference between a loop that works in a demo and one that would survive the Build Task's "a tool must be allowed to actually fail" requirement.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a MaxIterationsExceeded error, it's a kind of Exception

function run_agent(task, max_iterations) -> answer:
    messages = [the starting task]

    repeat up to max_iterations times:
        ask the model, with the tool available, using messages so far
        if the model asked for the tool:
            run the tool
            add the request and the result to messages
        else:
            this is the final answer -- stop and return it

    if we ran out of steps: raise MaxIterationsExceeded, don't just silently give
```

Here's almost the whole thing — fill in the tool-running part yourself:

```
# react_loop_practice.py – Intermediate section
def run_agent(task, max_iterations=5):
    messages = [{"role": "user", "content": task}]
    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[tool_schema],
        )
        message = response.choices[0].message
        if message.tool_calls:
            messages.append(message)
            # for each tool call: run it, append the {"role": "tool", ...} result
            # write this part yourself
        else:
            return message.content
    # write the "ran out of steps" case yourself
```

Intermediate Version

```
class MaxIterationsExceeded(Exception)

function run_agent(task: str, max_iterations: int = 5) -> str:
    messages: list[dict] = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        response = call the model with messages and tools
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            for call in message.tool_calls:
                result = run_tool(call)
                messages.append(
                    {"role": "tool", "tool_call_id": call.id, "content": result},
                )
        else:
            return message.content

    raise MaxIterationsExceeded(f"No answer after {max_iterations} steps")
```

```
# react_loop_practice.py – Intermediate section
def run_agent(task: str, max_iterations: int = 5) -> str:
    messages: list[dict] = [{"role": "user", "content": task}]

    for step in range(max_iterations):
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages,
            tools=[tool_schema],
        )
        message = response.choices[0].message

        if message.tool_calls:
            messages.append(message)
            # write the loop over message.tool_calls yourself: run each,
            # append its result as a "role": "tool" message
        else:
            return message.content

    # write the MaxIterationsExceeded raise yourself
```

Finish both marked parts yourself, then compare against the [Solution](#).

The piece worth seeing on its own before the full Solution — `run_tool()` catching a failure instead of letting it crash the loop, and looking the tool up from a dispatch table instead of hardcoding one:

```
# react_loop_practice.py – Intermediate section
import json

TOOLS_BY_NAME = {"get_weather": get_weather}

def run_tool(call) -> str:
    function = TOOLS_BY_NAME.get(call.function.name)
    if function is None:
        return f"Error: unknown tool {call.function.name}"
    try:
        args = json.loads(call.function.arguments)
        result = function(**args)
        return str(result)
    except Exception as e:
        return f"Error: {e}"
```

Notice this returns a *string* either way — a real result, or an error message — so the rest of `run_agent()` doesn't need to know or care which one it got; it just appends whatever `run_tool()` returns as the Observation, exactly like Hint 1 described. Also add a `steps` list that records every round's tool name, arguments, and result as it happens — you'll want this for the Build Task's step-log requirement, and for [14_debugging_lab](#) later. Try writing `run_agent()` so it returns both the final answer and this `steps` list (a small dataclass is a clean way to bundle the two) before checking the Solution.

Difference between Basic and Intermediate: Basic's pseudocode and near-complete code assume every tool call succeeds and nothing needs remembering afterward. Intermediate adds the real type contract, the exact message-passing shape the API requires, a `run_tool()` that turns an exception into an Observation instead of a crash and looks the tool up by name from a dispatch table, and a `steps` log that survives past the final answer.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)